UNIVERSITY OF CENTRAL FLORIDA

EEL6812 Neural Networks and Deep Learning

Assignment 1: Binary Classifier

Cory Brynds

College of Engineering and Computer Science
Department of Electrical and Computer Engineering
EEL6812 Neural Networks and Deep Learning

February 20, 2026

Contents

Contents i

List of Figures ii

List of Tables 1

1 Experiment 2

1.1 Questions 2

1.2 Network Evaluation 5

1.3 Hyperparameter Tuning 6

1.3.1 Number of Hidden Units 8

1.3.2 Learning Rate 8

1.3.3 Mini-Batch Size 9

1.3.4 Momentum Parameter 10

1.4 L2 Penalty Regularizer Tuning 10

List of Figures

1.1 Feedforward network architecture with annotations for parameter dimensions, assuming 15 hidden units. 2

1.2 Computation graph for backpropagation of feedforward network 3

1.3 Training loss for base model hyperparameters. 6

1.4 Training and testing misclassification rate for base model hyperparameters. 7

1.5 Training and testing error from network training using a sweep of hidden unit parameters. 8

1.7 Training and testing error from network training using a sweep of minibatch size parameters. 9

1.6 Training and testing error from network training using a sweep of learning rate parameters. 9

1.8 Training and testing error from network training using a sweep of momentum parameters. 10

1.9 Frobenius norm of $\mathbf{W}^{(1)}$ (left) and $\mathbf{W}^{(2)}$ (right) weight matrices over epochs across a sweep of L2 penalty regularizer parameters. 11

1.10 Training and testing error from network training using a sweep of L2 penalty regularizers. 11

List of Tables

1.1 Base hyperparameters used to initially train the classification network. . 6
1.2 Hyperparameter value ranges. 7
1.3 Hyperparameters identified as optimal through value range sweeps and their minimum achieved testing error rate. 7

1

Experiment

1.1 Questions

1. Describe the forward pass equations and the gradients of $\boldsymbol{W}^{(1)}$, $\boldsymbol{W}^{(2)}$, $\boldsymbol{b}^{(1)}$, and $\boldsymbol{b}^{(2)}$ computed using **backpropagation**. Clearly describe the dimensions of each model parameters $\boldsymbol{W}^{(1)}$, $\boldsymbol{W}^{(2)}$, $\boldsymbol{b}^{(1)}$, and $\boldsymbol{b}^{(2)}$.

   Forward pass equations:

$$a_1 = \mathrm{ReLU}(\mathbf{W}^{(1)}x + \mathbf{b}^{(1)})$$

$$a_2 = \sigma(\mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)})$$

$$J(\theta) = L_{CE} = -\left[y\log(\mathbf{a}^{(2)}) + (1-y)\log(1-\mathbf{a}^{(2)})\right]$$

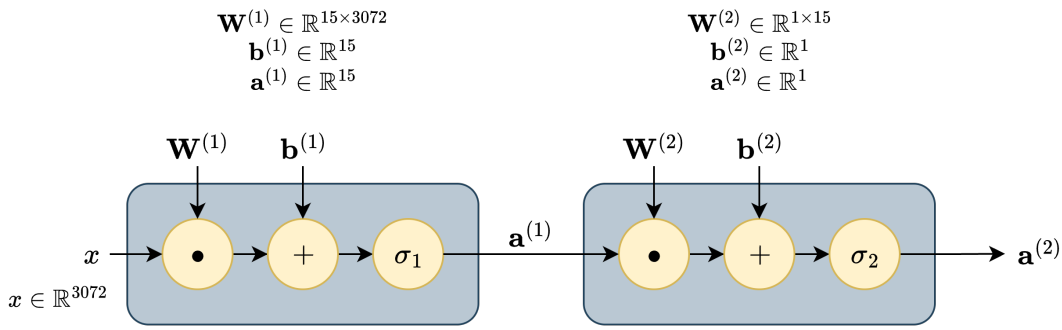

Figure 1.1: *Feedforward network architecture with annotations for parameter dimensions, assuming 15 hidden units.*

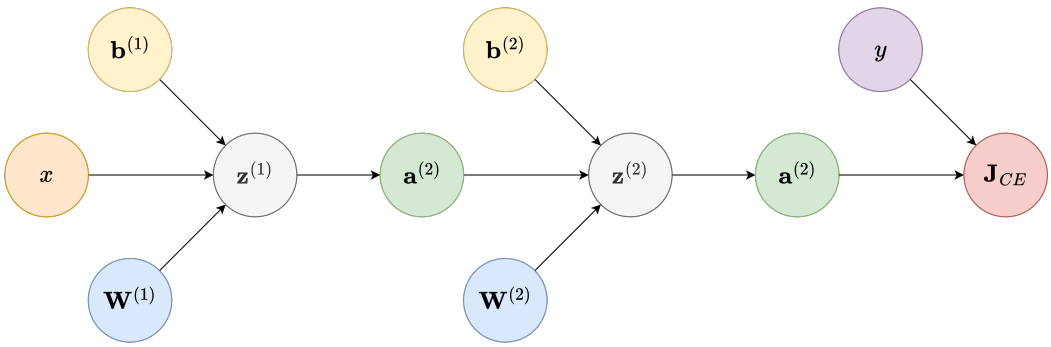

Figure 1.2: *Computation graph for backpropagation of feedforward network*

Partial gradients for each network parameter

Cross-Entropy Loss Gradient

$$\frac{\partial J}{\partial a^{(2)}} = -\left[\frac{y}{a^{(2)}} - \frac{1-y}{1-a^{(2)}}\right] = \frac{1-y}{1-a^{(2)}} - \frac{y}{a^{(2)}}$$

Sigmoid Gradient

$$\begin{aligned}
\frac{\partial a^{(2)}}{\partial z^{(2)}} &= \frac{d}{dz^{(2)}}\left(\frac{1}{1+e^{-z^{(2)}}}\right) = \frac{e^{-z^{(2)}}}{(1+e^{-z^{(2)}})^2} \\
&= \frac{1}{1+e^{-z^{(2)}}}\left(1 - \frac{1}{1+e^{-z^{(2)}}}\right) \\
&= a^{(2)}(1-a^{(2)})
\end{aligned}$$

$$\frac{\partial z^{(2)}}{\partial W^{(2)}} = a^{(1)}$$

$$\frac{\partial z^{(2)}}{\partial b^{(2)}} = 1$$

$$\frac{\partial z^{(2)}}{\partial a^{(1)}} = W^{(2)}$$

ReLU Gradient

$$\frac{\partial a^{(1)}}{\partial z^{(1)}} = \text{ReLU}'(z^{(1)}) = \begin{cases} 1: & z^{(1)} > 0 \\ 0: & \text{else} \end{cases}$$

$$\frac{\partial z^{(1)}}{\partial W^{(1)}} = x$$

$$\frac{\partial z^{(1)}}{\partial b^{(1)}} = 1$$

$$\frac{\partial z^{(1)}}{\partial x} = W^{(1)}$$

Final gradients for each network parameter

$$\begin{aligned}
\frac{dJ}{dz^{(2)}} &= \frac{\partial J}{\partial a^{(2)}} \frac{\partial a^{(2)}}{\partial z^{(2)}} = \left[\frac{1-y}{1-a^{(2)}} - \frac{y}{a^{(2)}}\right] [a^{(2)}(1-a^{(2)})] \\
&= (1-y)a^{(2)} - y(1-a^{(2)} \\
&= a^{(2)} - ya^{(2)} - y + ya^{(2)} \\
&= a^{(2)} - y \\
\frac{dJ}{dW^{(2)}} &= (a^{(2)} - y)a^{(1)} \\
\frac{dJ}{db^{(2)}} &= (a^{(2)} - y) \\
\frac{dJ}{da^{(1)}} &= (a^{(2)} - y)W^{(2)} \\
\frac{dJ}{dz^{(1)}} &= \text{ReLU}'(z^{(1)})(a^{(2)} - y)W^{(2)} \\
\frac{dJ}{dW^{(1)}} &= \text{ReLU}'(z^{(1)})(a^{(2)} - y)[W^{(2)}]^{\intercal} x \\
\frac{dJ}{db^{(1)}} &= \text{ReLU}'(z^{(1)})(a^{(2)} - y)W^{(2)}
\end{aligned}$$

For gradient updates, gradients are averaged across samples within each mini-batch.

2. Write the cross-entropy loss function used for optimization.

$$L_{CE} = -\sum_{i=1}^{N} \left[y \log(\hat{y}) + (1-y) \log(1-\hat{y}) \right]$$

where y is the true data label and $\hat{y} = \mathbf{a}^{(2)}$ is the predicted probability.

3. Regularize the cross-entropy loss function using weight decay (i.e., L2 regular-

ization of the parameters). Write the total loss loss function $L_{CE} + \lambda L_{reg}$.

$$L_{reg} = \frac{1}{2}\left[\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_2^2\right]$$

$$L = L_{CE} + \lambda L_{reg}$$

$$= -\sum_{i=1}^{N}\left[y\log(\hat{y}) + (1-y)\log(1-\hat{y})\right] + \frac{\lambda}{2}\left[\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_2^2\right]$$

Biases are not included in the L2 regularization term. According to ML literature, weight decay has a much greater impact on network learning than bias decay, and large weights will make a network more sensitive to inputs than large biases.

4. Write the momentum updates used for the mini-batch stochastic gradient descent algorithm.

$$\mathbf{m}^{(t+1)} = \mu\mathbf{m}^{(t)} - \alpha\Delta_\theta J(\theta^{(t)})$$
$$\theta^{(t+1)} = \theta^{(t)} + \mathbf{m}^{(t+1)}$$

where $\mu \in [0, 1]$ (0.8 for the base hyperparameters in this project) and $\mathbf{m}$ is the momentum term in the same shape as θ that is initialized to zero. Here, the gradient is a mini-batch gradient, and θ refers to the weights and biases of the network.

1.2 Network Evaluation

Table 1.1 shows the base hyperparameters used to train the neural network. All specified hyperparameters in the assignment were used, and the number of epochs was set to be 100 based on empirical testing of when loss saturates. From Figure 1.4, after 100 epochs, training error reaches a minimum of 12.5%, while testing error reaches a minimum of around 18% after 80 epochs. Training loss has a roughly exponential decay, reaching 0.3 after 100 epochs.

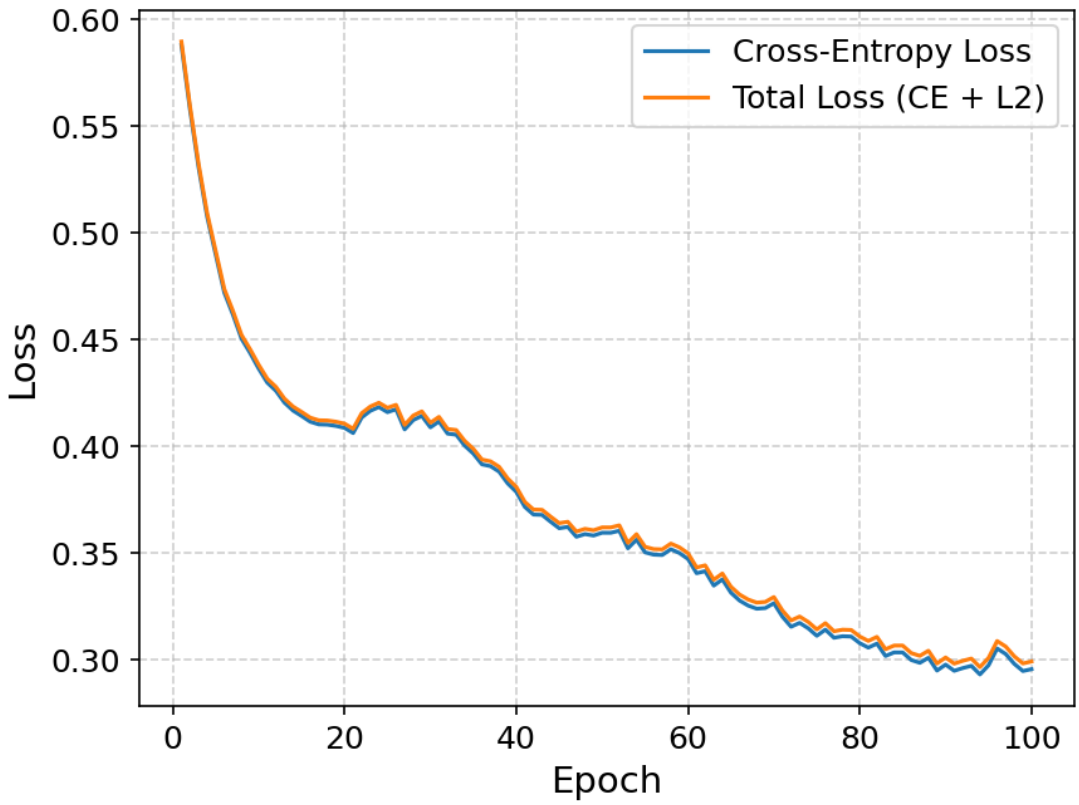

Figure 1.3: *Training loss for base model hyperparameters.*

Hyperparameter	Value
Number of epochs	100
Number of hidden units	15
Learning rate	0.01
Mini-batch size	200
Momentum parameter	0.8
L2 penalty regularizer	0.0001

Table 1.1: *Base hyperparameters used to initially train the classification network.*

1.3 Hyperparameter Tuning

Table 1.2 shows the value ranges swept for each hyperparameter to generate the plots in Figures 1.5- 1.8. A summary of the optimal hyperparameters for each section and the minimum test error they produce is available in Table 1.3.

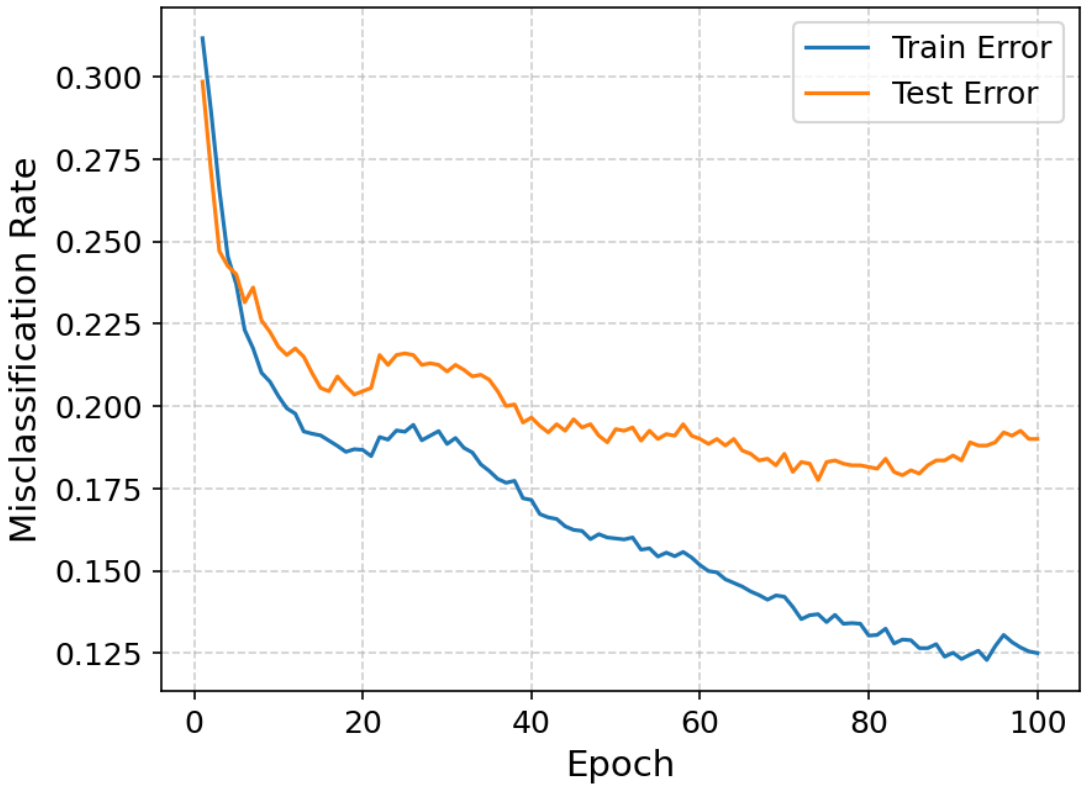

Figure 1.4: *Training and testing misclassification rate for base model hyperparameters.*

Hyperparameter						
Number of hidden units	5	10	15	20	25	30
Learning rate	0.001	0.01	0.03	0.05	0.1	
Mini-batch size	10	50	100	200	500	
Momentum parameter	0	0.5	0.8	0.9		

Table 1.2: *Hyperparameter value ranges.*

Hyperparameter	Optimal Value	Min. Testing Error (%)
Number of hidden units	15	0.19
Learning rate	0.01	0.195
Mini-batch size	100	0.18
Momentum parameter	0.8	0.185
L2 penalty regularizer	0.0001	0.175

Table 1.3: *Hyperparameters identified as optimal through value range sweeps and their minimum achieved testing error rate.*

1.3.1 Number of Hidden Units

For the number of hidden units, 15 consistently yielded the lowest testing error, although 10 and 30 hidden units were close contenders for minimizing testing error in the later epochs. As seen in Figure 1.5, too few hidden units (5 units) leads to underfitting (i.e. high misclassification rate). More than 15 hidden units offers negligible reduction in testing error, but it will increase the cost in training and evaluating the network.

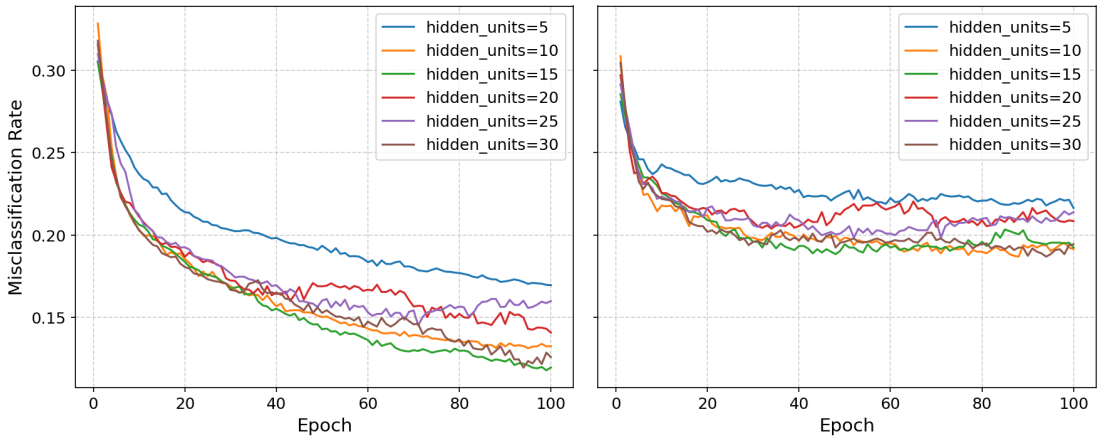

Figure 1.5: *Training and testing error from network training using a sweep of hidden unit parameters.*

1.3.2 Learning Rate

For the learning rate, a learning rate of 0.01 yielded the lowest testing error. In some training runs, the testing error for a learning rate of 0.1 approached 50%, a random chance for picking the right class. This is because the model's parameters were being updated too sharply during training. Similarly for a learning rate of 0.001, the model's weights are being updated too slowly each epoch, which takes the network longer to converge to an optimal solution. A learning rate of 0.01 is a balance between the two extremes, smoothing out gradient updates yet converging in a reasonable amount of epochs.

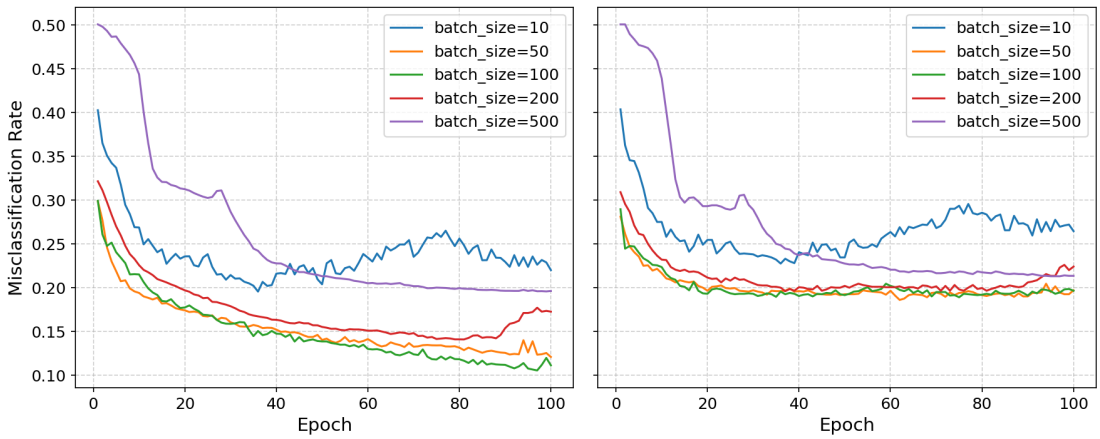

Figure 1.7: *Training and testing error from network training using a sweep of mini-batch size parameters.*

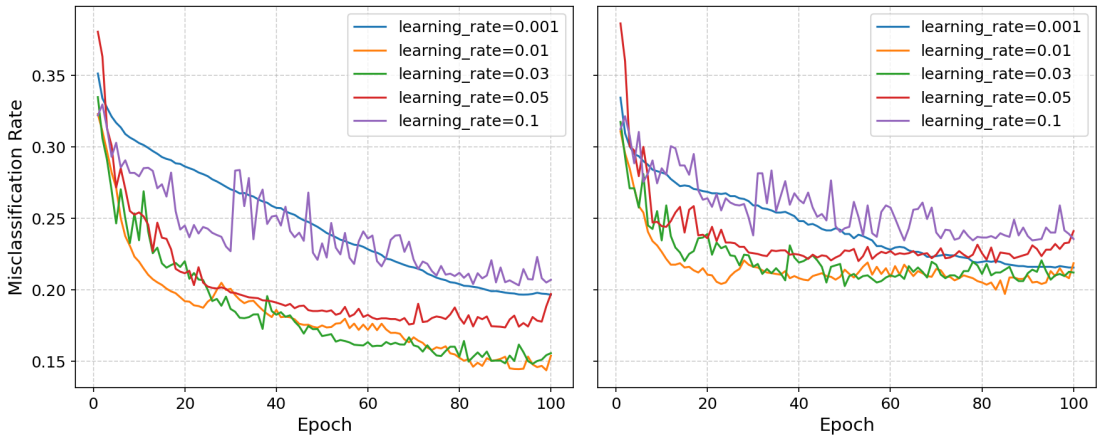

Figure 1.6: *Training and testing error from network training using a sweep of learning rate parameters.*

1.3.3 Mini-Batch Size

The mini-batch size contributing to the lowest testing error rate is 100, shown in the right plot of Figure 1.7. Too small of a mini batch size (10) causes the gradient updates to be noisier and less stable. As shown in the plots, it may even diverge and cause error to steadily increase in later epochs. Too large of a batch size (500) results in too few updates per epoch (since the number of training steps each epoch is num samples / num batches), which means that it will take longer for the network to learn. With a batch size of 100, there are a good amount of gradient updates per epoch, and the gradient update each step is more stable.

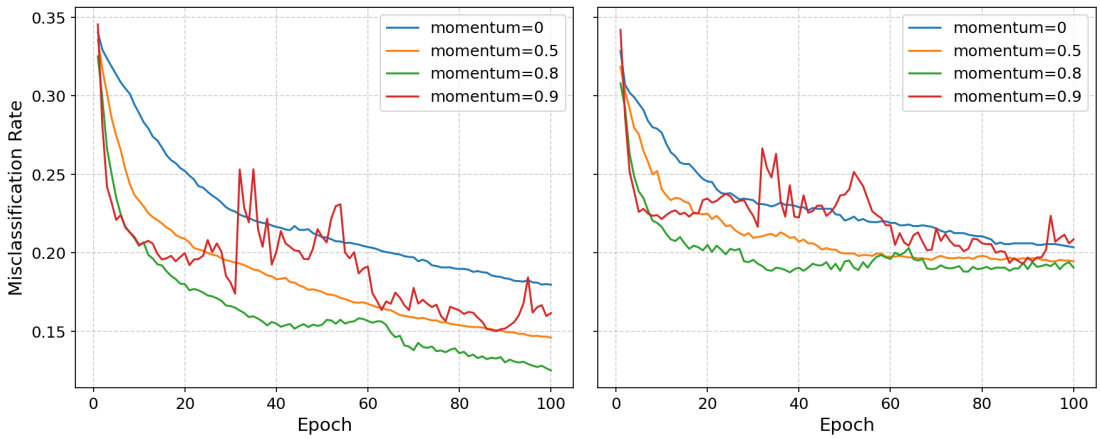

Figure 1.8: *Training and testing error from network training using a sweep of momentum parameters.*

1.3.4 Momentum Parameter

For minimizing testing error, a momentum parameter of 0.8 was observed to perform the best almost universally. In early epochs, it lead to the sharpest increase in model accuracy on training and testing data. Too much momentum, as seen in the plot for 0.9, will contribute to sharp changes in model parameters over time, and no momentum (normal gradient update) will converge to a lower testing error more slowly, although it does still converge. Again, as is the trend in the other hyperparameters, the ideal parameter is often a balance between two extremes of stabilizing the gradient updates without slowing down the speed of network convergence to an optimal solution.

1.4 L2 Penalty Regularizer Tuning

In this section, the L2 penalty is swept across a parameter range $(0, 0.01, 0.001, 0.0001)$.

Across epochs, the magnitude of the $\mathbf{W}^{(1)}$ weight matrix norm increases linearly, with the exception of $\lambda = 0.01$, which decreases exponentially over epochs. This can be interpretted as pulling the weights towards zero with larger gradient updates each step. This manifests as a larger overall training error than the other parameters, although it does converge to roughly the same testing error. The magnitude of the norm curve decreases with increasing L2 penalty. For the $\mathbf{W}^{(2)}$ weight matrix, the magnitude of the norm increases over epochs. A decreasing L2 penalty has the general effect of increasing the magnitude of the L2 norm, except for the case of $\lambda = 0$.

The L2 penalty $\lambda = 0.001$ leads to the lowest testing error rate, followed closely by $\lambda = 0.0001$. From training error, $\lambda = 0.0001$ performs the best. Looking at the trends

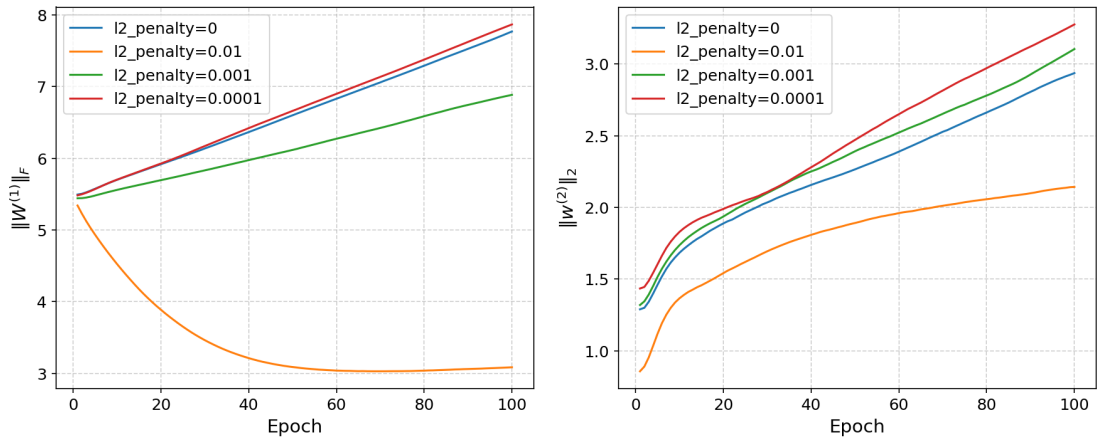

Figure 1.9: *Frobenius norm of* $\mathbf{W}^{(1)}$ *(left) and* $\mathbf{W}^{(2)}$ *(right) weight matrices over epochs across a sweep of L2 penalty regularizer parameters.*

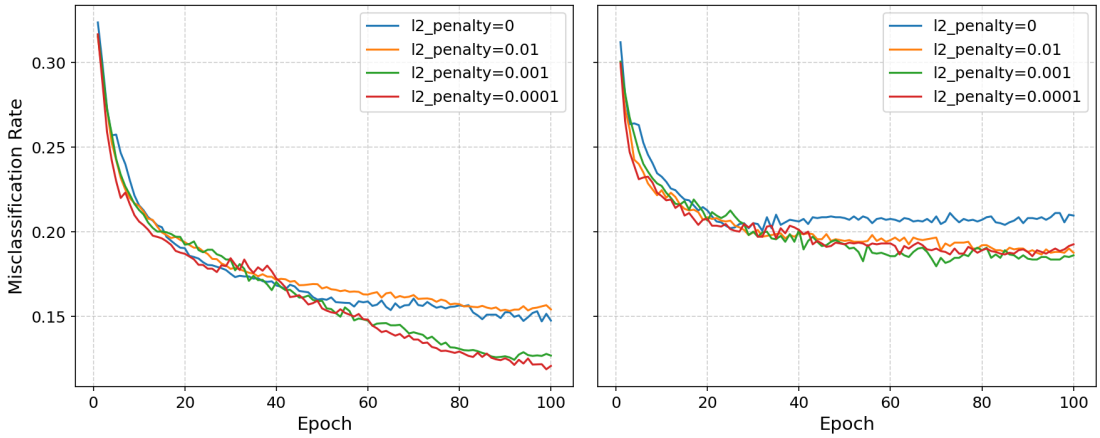

Figure 1.10: *Training and testing error from network training using a sweep of L2 penalty regularizers.*

in their weight norms, they both follow a similar pattern, although the weights for $\lambda = 0.001$ increase at a slower rate.